ITLSD 601

ALGORITHMS AND DATA STRUCTURES USING C

Rewritten CAT Questions + Correct Answers + Practical C Programs

Based directly on the photographed CAT paper provided.

Important: The practical electricity-billing solution below interprets the rates as progressive slabs: first 50 units, next 100, next 100, then units above 250.

SECTION A — THEORETICAL CAT

10 marks • Multiple choice, True/False and open-ended questions

SECTION B — PRACTICAL CAT

30 marks • Seven C programming problems

Exam focus: C functions, algorithms, flowcharts, pointers, arrays, loops, operators, conditions, switch-case and practical C programs.

SECTION A — THEORETICAL CAT

Rewritten questions with answers

1. Which of the following is a correct syntax for a C function?

A. function int() {}

B. int function() {}

C. func int()

D. int = function() {}

Answer: B. int function() {} — a C function declaration/definition has a return type, function name, parentheses and a function body.

2. What is an algorithm?

A. A flowchart

B. A programming language

C. Step-by-step instructions to solve a problem

D. A compiler

Answer: C. Step-by-step instructions to solve a problem.

3. Which symbol is used for a decision in a flowchart?

A. Rectangle

B. Circle

C. Diamond

D. Parallelogram

Answer: C. Diamond. The diamond represents a decision/condition.

4. The main() function in C:

A. Is optional

B. Can never return a value

C. Is where program execution starts

D. Is defined automatically by the compiler

Answer: C. It is where program execution starts.

5. Which of the following is a valid keyword in C?

A. integer

B. var

C. int

D. number

Answer: C. int.

6. What symbol is used to start a single-line comment in C?

A. //

B. /*

C. #

D. *

Answer: A. //

7. Which operator is used to compare two values for equality?

A. =

B. ==

C. !=

D. +=

Answer: B. ==

8. Which loop checks the condition after executing the block of code at least once?

A. for

B. while

C. do...while

D. if

Answer: C. do...while

TRUE OR FALSE — 4 marks

A. Every algorithm must terminate after a finite number of steps.

Answer: TRUE

B. The scanf() function is used for output in C.

Answer: FALSE — scanf() is used to receive/input data; printf() is used for formatted output.

C. A brute-force algorithm tries every possible solution.

Answer: TRUE

D. C supports recursion.

Answer: TRUE

E. Arrays in C can store different types of data in the same array.

Answer: FALSE — a normal C array stores elements of the same declared type.

F. The char data type can store different types of data.

Answer: FALSE — char is used for character/integer character-code values, not arbitrary data types.

G. The scanf() function is used to display output to the user.

Answer: FALSE — printf() displays output; scanf() reads input.

OPEN-ENDED QUESTIONS — 2 MARKS

1. Explain the purpose of a loop in C programming.

Answer: A loop repeatedly executes a block of code while a condition is satisfied or for a specified number of iterations.

2. Describe what a pointer is in simple terms.

Answer: A pointer is a variable that stores the memory address of another variable.

3. Describe two characteristics of a good algorithm.

Answer: Any two: it should be clear/unambiguous, have well-defined input and output, be finite, be feasible/practical, and be language independent.

4. Define an array in programming.

Answer: An array is a collection of elements of the same data type stored under one name and accessed using indexes.

Quick memory check

Loop: repeat code. Pointer: stores an address. Algorithm: finite step-by-step solution. Array: same-type elements accessed by index.

SECTION B — PRACTICAL CAT

Rewritten questions and complete C solutions

1. Positive, negative or zero

Write a C program to check whether a number is negative, positive or zero using if-else.

```
#include <stdio.h>

int main(void)
{
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    if (number > 0)
    {
        printf("Positive\n");
    }
    else if (number < 0)
    {
        printf("Negative\n");
    }
    else
    {
        printf("Zero\n");
    }

    return 0;
}
```

Key point: Logic: number > 0 → Positive; number < 0 → Negative; otherwise → Zero.

2. Print natural numbers in reverse

Write a C program to print all natural numbers in reverse, from n to 1, using a while loop.

```
#include <stdio.h>

int main(void)
{
    int n;

    printf("Enter n: ");
    scanf("%d", &n);

    while (n >= 1)
    {
        printf("%d ", n);
        n--;
    }

    printf("\n");
    return 0;
}
```

Key point: Example: if n = 5, output is 5 4 3 2 1.

3. Even or odd using switch-case

Write a C program to check whether a number is even or odd using switch-case.

```
#include <stdio.h>

int main(void)
{
    int number;

    printf("Enter a number: ");
    scanf("%d", &number);

    switch (number % 2)
    {
        case 0:
            printf("Even\n");
            break;
```

```
            case 1:
            case -1:
                printf("Odd\n");
                break;

            default:
                printf("Invalid result\n");
        }

        return 0;
    }
```

Key point: Logic: an even number has remainder 0 when divided by 2; an odd number has remainder 1 (or -1 for a negative odd integer).

4. Electricity bill calculation

Write a C program to input electricity-unit consumption and calculate the total electricity bill according to these conditions: first 50 units at Rs. 0.50/unit; next 100 units at Rs. 0.75/unit; next 100 units at Rs. 1.20/unit; units above 250 at Rs. 1.50/unit. Add a 20% surcharge to the bill.

Interpretation used: progressive/slab billing. Each block of units is charged at its own rate.

```
#include <stdio.h>

int main(void)
{
    float units, bill, surcharge, total;

    printf("Enter electricity units: ");
    scanf("%f", &units);

    if (units <= 50)
    {
        bill = units * 0.50f;
    }
    else if (units <= 150)
    {
        bill = (50 * 0.50f) +
               ((units - 50) * 0.75f);
    }
    else if (units <= 250)
    {
        bill = (50 * 0.50f) +
               (100 * 0.75f) +
               ((units - 150) * 1.20f);
    }
    else
    {
        bill = (50 * 0.50f) +
               (100 * 0.75f) +
               (100 * 1.20f) +
               ((units - 250) * 1.50f);
    }

    surcharge = bill * 0.20f;
    total = bill + surcharge;

    printf("Electricity charge = Rs. %.2f\n", bill);
    printf("Surcharge = Rs. %.2f\n", surcharge);
    printf("Total bill = Rs. %.2f\n", total);

    return 0;
}
```

Exam trap: Do not charge all units at the rate of the final slab. Charge each slab progressively, then add 20% of the calculated bill.

5. Uppercase or lowercase

Write a C program to check whether a character is uppercase or lowercase.

```
#include <stdio.h>

int main(void)
{
    char ch;

    printf("Enter a character: ");
    scanf(" %c", &ch);

    if (ch >= 'A' && ch <= 'Z')
    {
        printf("Uppercase\n");
    }
    else if (ch >= 'a' && ch <= 'z')
    {
        printf("Lowercase\n");
    }
    else
    {
        printf("Not an alphabetic character\n");
    }

    return 0;
}
```

Key point: Use the character ranges A-Z and a-z.

6. Area of a circle

Write a C program to calculate and display the area of a circle. Prompt the user for the radius. Use area = π × r × r and π ≈ 3.14159.

```
#include <stdio.h>

int main(void)
{
    float radius, area;
    const float PI = 3.14159f;

    printf("Enter radius: ");
    scanf("%f", &radius);

    area = PI * radius * radius;

    printf("Area = %.2f\n", area);

    return 0;
}
```

Key point: Formula: area = πr^2.

7. Age-group classification

Write a C program that takes a person's age as input and classifies the person using if statements: Infant 0-2 years; Toddler 3-5 years; Child 6-12 years; Teenager 13-19 years; Adult 20 years and above.

```
#include <stdio.h>

int main(void)
{
    int age;

    printf("Enter age: ");
    scanf("%d", &age);

    if (age < 0)
    {
        printf("Invalid age\n");
    }
    else if (age <= 2)
    {
        printf("Infant\n");
    }
```

```
    else if (age <= 5)
    {
        printf("Toddler\n");
    }
    else if (age <= 12)
    {
        printf("Child\n");
    }
    else if (age <= 19)
    {
        printf("Teenager\n");
    }
    else
    {
        printf("Adult\n");
    }

    return 0;
}
```

Key point: Check the ranges from the smallest age upward so that every valid age falls into exactly one group.

LAST-MINUTE EXAM REVISION

Memorize these patterns before the CAT

Topic	What to remember
Function	return_type function_name(parameters) { ... }
Algorithm	Finite, clear step-by-step instructions to solve a problem.
Decision flowchart	Diamond
Process flowchart	Rectangle
Input/Output	Parallelogram
Start/Stop	Oval/terminator
Single-line comment	//
Equality comparison	==
Assignment	=
AND / OR / NOT	&& / || / !
Loop after condition	do...while
Pointer	Stores an address
Array	Same-type elements; index starts at 0
Input	scanf()
Output	printf()
Even test	number % 2 == 0
Circle area	PI * radius * radius

Programming checklist

1. Include #include <stdio.h> when using printf/scanf.
2. Start execution in main().
3. Use & with scanf for ordinary variables, e.g. scanf("%d", &x).
4. End C statements with ;.
5. For if/else questions, write the ranges carefully.
6. For loops, update the loop variable so the condition can eventually become false.
7. For switch-case, remember break.
8. For practical questions, test your program with at least one normal example and one boundary value.

Boundary values worth testing: 0, 1, 2, 3, 5, 6, 12, 13, 19, 20, 50, 51, 150, 151, 250, 251.